Business Process Management Solution

HIGH-LEVEL DESIGN & LOW-LEVEL DESIGN

Version: 1.0 | **Date:** February 2026 | **Status:** Released

Environment: Docker Compose (Self-Hosted)

Classification: Internal — Technical

CONFIDENTIAL

Table of Contents

1. Executive Summary

The **BPM Portal** is a full-stack, cloud-ready Business Process Management platform built to digitalise, automate, and monitor enterprise workflows. It provides a unified environment for modelling processes in BPMN 2.0, executing them at runtime, managing ITIL service cases, enforcing multi-level approvals, and integrating with external systems — all governed by role-based access control and a complete audit trail.

Primary Goals

- Eliminate manual, paper-based workflows
- Provide real-time process visibility & SLA enforcement
- Centralise approval chains across IT, Finance & Operations
- Enable no-code process design via a browser-based modeler
- Deliver ITIL-aligned case management (Incident, Change, Problem, Request)

Key Outcomes

- 7 independent NestJS microservices, fully Dockerised
- BPMN 2.0 process execution without an external engine
- Real-time event streaming via Kafka (KRaft, no Zookeeper)
- Zero-trust API gateway with Keycloak OIDC/JWT
- Multi-tenant architecture with tenant isolation
- Live analytics dashboard with SLA compliance tracking

Solution at a Glance

Dimension	Detail
Architecture Style	Microservices, Event-Driven, RESTful API
Runtime	Docker Compose (15 containers)
Backend Framework	NestJS (Node.js / TypeScript)
Frontend	React 18 + TypeScript + Material-UI + Vite
Database	PostgreSQL 15 (single shared instance, 6 migration sets)
Message Broker	Apache Kafka 3.x — Bitnami KRaft mode
Auth	Keycloak 24 — OIDC / JWT (RS256)
Storage	MinIO (S3-compatible) for file attachments
BPMN Modeler	bpmn-js (Camunda-compatible + Flowable import)
Monitoring	Prometheus + Grafana
API Spec	Swagger / OpenAPI (auto-generated per service)

2. Solution Overview

2.1 Business Objectives

Organisations frequently struggle with fragmented, email-driven processes that are invisible to management, difficult to audit, and impossible to improve systematically. The BPM Portal addresses this by providing:

- **Process Design:** A drag-and-drop BPMN 2.0 studio for creating and publishing workflows without writing code.
- **Process Execution:** An embedded token-based execution engine that drives user tasks, service tasks, and gateway branching at runtime.
- **Case Management:** ITIL-aligned case lifecycle management for incidents, problems, change requests, service requests, and alarms.
- **Approval Governance:** Configurable multi-step approval chains with conditional routing, delegation, and escalation.
- **Integration:** Outbound connectors for REST APIs, webhooks, Kafka producers, and scheduled cron jobs — enabling automation without custom code.
- **Visibility:** Real-time analytics, bottleneck detection, SLA compliance monitoring, and a complete append-only audit trail.

2.2 Scope & Capabilities

Process Management

- BPMN 2.0 visual designer with Properties Panel
- Import from Flowable / Camunda XML
- Draft → Active → Archived lifecycle
- Start/suspend/resume/terminate instances
- Variable propagation through gateways
- SLA timers on user tasks (5-min breach check)

Case Management

- 5 ITIL case types with auto-numbered IDs (INC, PRB, CHG, REQ, ALM)
- Full state machine (new → open → resolved → closed)
- SLA due-date enforcement by type + priority
- Team & individual assignment
- Linked process instances per case
- Work notes (internal/public comments)

Approvals

- Configurable multi-step policies
- Condition-based step activation
- Hierarchy-based (N levels up) routing
- Role-based routing (any user in role)
- Delegation & out-of-office support
- Parallel approval steps

Integration Hub

- REST outbound connector (any HTTP method)
- Webhook receiver (inbound events)
- Kafka producer connector
- Cron scheduler with variable interpolation
- Execution log per connector
- Manual test execution from UI

Analytics

- KPI cards: instances, tasks, SLA breaches
- Bar chart: instances by definition
- Line chart: started vs completed over time
- Bottleneck table: queued tasks by node
- SLA compliance table with progress bars
- User workload distribution

Notifications

- In-app notification bell with unread count
- Email channel support
- Handlebars template engine
- Kafka-triggered (auto on task/case events)
- Mark-all-read from Work Inbox
- Per-entity notification context

2.3 Technology Stack

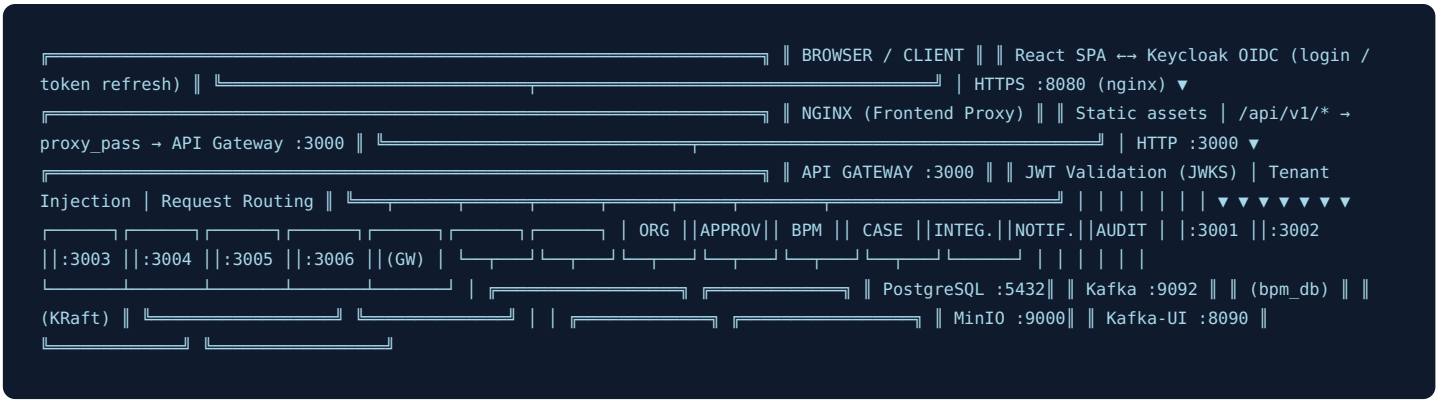
Layer	Technology	Version	Purpose
Frontend	React + TypeScript	18.x	SPA UI
Frontend UI	Material-UI (MUI)	5.x	Component library
Frontend Build	Vite	5.x	Fast bundler
BPMN Editor	bpmn-js	latest	BPMN 2.0 modeler & viewer
Charts	Recharts	2.x	Analytics visualisations
Backend	NestJS + TypeScript	10.x	Microservice framework
Runtime	Node.js	20.x LTS	JavaScript runtime
Database	PostgreSQL	15.x	Primary data store
DB Driver	pg (node-postgres)	8.x	Raw SQL, parameterized queries
Message Broker	Apache Kafka (KRaft)	3.x	Async event streaming
Kafka Client	KafkaJS	2.x	Producer & consumer
Auth Server	Keycloak	24.x	OIDC / OAuth2 provider
Auth Library	passport-jwt + jwks-rsa	—	JWT validation via JWKS
Object Storage	MinIO	latest	S3-compatible file storage

Metrics	Prometheus + Grafana	latest	Infrastructure monitoring
Reverse Proxy	Nginx	1.25	Frontend serving + SPA routing
Container	Docker + Compose	v2	Orchestration & isolation
Template Engine	Handlebars	4.x	Notification rendering

3. High-Level Design (HLD)

3.1 System Architecture

The BPM Portal follows a **microservices architecture** with a dedicated API Gateway serving as the single entry point. Services communicate synchronously via HTTP (through the gateway or direct service-to-service) and asynchronously via Kafka topics. All services share a single PostgreSQL database with schema isolation enforced by tenant IDs.



3.2 Service Decomposition

Service	Port	Domain Responsibility	Key Dependencies
api-gateway	3000	Single entry point — JWT validation, tenant injection, request proxying, Swagger aggregation, audit logging	Keycloak (JWKS), all downstream services, Kafka
org-service	3001	Organisational structure — tenants, org units (hierarchical materialized path), users, roles, positions, delegation	PostgreSQL
approval-service	3002	Approval workflow engine — policies (multi-step, conditional), instances, decisions, delegations	PostgreSQL, Kafka, Org-Service (manager resolution)
bpm-orchestrator	3003	Core BPM engine — BPMN definitions, process execution, task management, analytics, SLA scheduling	PostgreSQL, Kafka
case-service	3004	ITIL case management — incident/change/problem/request/alarm lifecycle, SLA enforcement, comments, timeline	PostgreSQL, Kafka
integration-hub	3005	Outbound integrations — REST, webhook, Kafka producer, cron connectors; execution logs; Kafka consumer for service tasks	PostgreSQL, Kafka, external HTTP endpoints
notification-service	3006	Notification delivery — templates, in-app and email channels, Kafka-driven triggers, read status	PostgreSQL, Kafka

3.3 Data Flow Architecture

Synchronous Flow (REST)

The primary path for user-initiated actions:

```
Browser → Nginx :8080 → /api/v1/* → API Gateway :3000
  → [JWT Validation] → [Tenant Extraction] → [Proxy Forward]
  → Downstream Service → PostgreSQL → Response → Browser
```

Process Execution Flow

When a user completes a task, the process engine advances automatically:

```
POST /api/v1/tasks/{id}/complete
  → BPM Orchestrator: task.service.ts
  → UPDATE tasks SET status='completed', outcome=…
  → process-instance.service.ts: onTaskCompleted()
  → parseBpmn(instance.bpmn_xml)
  → getOutgoingFlows(nodeId)
  → advance() : evaluate conditions / create next tasks
  → Kafka: bpm.task.completed  → Notification Service (email/in-app)
  → Kafka: bpm.task.created    → Notification Service (new task alert)
```

Case Creation with Process Linkage

```
POST /api/v1/cases  (title, type, priority, assigned_team_id, context)
  → case-service: create()  → INSERT cases RETURNING *
  → Kafka: bpm.case.created → Notification Service + Integration Hub
  POST /api/v1/processes/instances  (definitionId, caseId, businessKey, variables)
  → bpm-orchestrator: start()  → INSERT process_instances
  → parseBpmn() → advance() → createTask() → Kafka: bpm.task.created
```

Asynchronous Event Flow

```
Event Producer          Kafka Topic                Consumer(s)
  ──────────────────────────────────────────────────────────────
  BPM Orchestrator  →  bpm.task.created         →  Notification Svc
  BPM Orchestrator  →  bpm.task.sla_breach      →  Notification Svc
  BPM Orchestrator  →  bpm.service.task         →  Integration Hub
  Approval Svc      →  bpm.approvals            →  Notification Svc
  Case Svc          →  bpm.case.created         →  Notification Svc, Integration Hub
  Case Svc          →  bpm.case.sla_breach      →  Notification Svc
```

3.4 Security Architecture

Zero-Trust Perimeter

Every request entering the API Gateway is validated against Keycloak's JWKS endpoint before being proxied downstream. Internal service-to-service calls bypass public auth but rely on network isolation (Docker internal network).

Layer	Mechanism	Details
Identity Provider	Keycloak 24	OAuth2 / OIDC server, realm "bpm", RS256 signed JWTs, PKCE flow for SPA
Token Validation	JWKS (passport-jwt + jwks-rsa)	Public key fetched from `http://keycloak:8080/realms/bpm/protocol/openid-connect/certs`, cached, rate-limited
Tenant Isolation	TenantInterceptor	Extracts tenantId from JWT claim → injects as `X-Tenant-ID` header. Every DB query filters by tenant_id.
User Identity	X-User-ID header	Gateway passes `req.user.sub` (Keycloak UUID) as `X-User-ID`. Services resolve to internal UUID via `keycloak_id` lookup.
Role-Based Access	Realm roles	Keycloak `realm_access.roles` array in JWT payload. Roles: admin, manager, requester, finance_controller, cab_member, it_engineer
HTTPS	Keycloak TLS + Nginx	Keycloak exposed on :8443 (HTTPS). Production: terminate TLS at load balancer.
Audit Trail	PostgreSQL RULE	Append-only audit_log table — UPDATE and DELETE are blocked by database rules. Actor ID, before/after state, IP address recorded.
HTTP Headers	helmet.js	All NestJS services use helmet for security headers (CSP, HSTS, X-Frame-Options).

3.5 Integration Architecture

External integrations are managed entirely through the **Integration Hub** service, which abstracts connectivity into reusable connector configurations stored in the database.

Connector Types

- **REST** — Outbound HTTP (GET/POST/PUT/PATCH/DELETE). Supports headers, body, authentication, variable interpolation in URLs and payloads using `{{variableName}}` syntax.
- **Webhook** — Inbound event receiver. Validates incoming payloads and routes to internal processing.
- **Kafka Producer** — Publish messages to Kafka topics from process context.
- **Cron** — Scheduled execution via Node.js timers (loaded at startup from DB for active connectors).

⚡ Kafka Service-Task Bridge

When the BPMN engine encounters a `serviceTask` node, it emits a `bpm.service.task` event containing:

- **serviceType** — maps to a connector category
- **config** — connector-specific parameters
- **variables** — current process variables

The Integration Hub consumes this event and executes the appropriate connector, enabling fully automated service task execution within BPMN flows.

3.6 Observability & Monitoring

Tool	Port	What it Monitors
Prometheus	9090	Scrapes metrics from all NestJS services (`/metrics` endpoint), Kafka, PostgreSQL
Grafana	3300	Dashboards for request rates, error rates, DB connections, Kafka lag
Kafka UI	8090	Topic browser, consumer group lag, message inspector
MinIO Console	9001	Object storage browser, bucket management
Audit Log	(API)	Business-level audit: who did what, when, with before/after state
Gateway Kafka	(Kafka)	Every API request logged to `bpm.gateway.requests` with requestId, duration, status code

4. Low-Level Design (LLD)

4.1 Database Schema

> **i Design Principle — Shared Database, Logical Isolation**
>
> All services share a single PostgreSQL instance (`bpm_db`). Multi-tenancy is enforced at the application layer: every table carries a `tenant_id UUID NOT NULL` column that acts as a partition key. All queries include a `WHERE tenant_id=$1` predicate. Row-level security is not used; isolation is enforced by the application tier.

Migration 001 — Tenants & Organisation

Table	Key Columns	Notes
tenants	id, name, slug, settings JSONB	Root of multi-tenancy; slug used in URL routing
org_units	id, tenant_id, name, type, parent_id, path (VARCHAR materialized path), manager_id	Hierarchy via materialized path: `/parent_id/child_id/` . Types: company → division → department → section → team
positions	id, org_unit_id, name, level, is_manager	Job titles within an org unit
roles	id, tenant_id, name, key, permissions JSONB	RBAC — permissions array of action strings
users	id, tenant_id, keycloak_id, email, first_name, last_name, username, active	keycloak_id links to Keycloak realm user UUID for actor resolution
user_org_assignments	user_id, org_unit_id, position_id, is_primary, effective_from/to	A user can belong to multiple org units; primary flag for default team
user_roles	user_id, role_id, granted_by, granted_at	Many-to-many role assignments

Migration 002 — Approval Engine

Table	Key Columns	Notes
approval_policies	id, tenant_id, name, entity_type, steps JSONB, conditions JSONB, created_by	Steps: array of {index, type: hierarchy|role|user, level, roleId, parallel, condition}
approval_instances	id, policy_id, entity_type, entity_id, status (pending/approved/rejected/cancelled/escalated), requester_id, resolved_steps JSONB	Snapshot of steps at creation time to prevent policy mutation affecting in-flight approvals
approval_step_decisions	instance_id, step_index, approver_id, decision (approved/rejected/delegated/escalated), comments, decided_at	Immutable per-step decisions
delegations	delegator_id, delegate_id, tenant_id, start_date, end_date, active, reason	Out-of-office delegation; approval resolver checks active delegations first

Migration 003 — BPM Process Engine

Table	Key Columns	Notes
process_definitions	id, tenant_id, name, slug (UNIQUE per tenant), bpmn_xml TEXT, config JSONB, version INT, status (draft/active/archived)	bpmn_xml stores the full BPMN 2.0 XML. Only `active` definitions can be started.
process_instances	id, tenant_id, definition_id, business_key, status (active/completed/suspended/terminated), variables JSONB, current_node_id,	variables JSONB accumulates task outputs;

	initiator_id, case_id, started_at, completed_at	merged at each gateway evaluation
tasks	id, tenant_id, process_instance_id, node_id, name, type, status (pending/in_progress/completed/cancelled/escalated), assignee_id, candidate_groups JSONB, variables JSONB, due_at, sla_breached, outcome, claimed_at, completed_at, comments JSONB	candidate_groups can hold group names or individual user IDs for flexible assignment

Migration 004 — Case Management

Table	Key Columns	Notes
cases	id, tenant_id, case_number (INC1001), type (incident/problem/change/request/alarm), title, status, priority, impact, urgency, requester_id, assignee_id, assigned_team_id, process_instance_id, context JSONB, sla_due_at, breached, resolved_at, closed_at	ITIL-aligned. SLA computed from type × priority matrix. context JSONB stores process-derived form data.
case_sequences	tenant_id, prefix, next_val	ON CONFLICT DO UPDATE to atomically increment per-prefix counter (INC, PRB, CHG, REQ, ALM)
case_comments	case_id, tenant_id, author_id, body, internal (bool)	internal=true for agent-only work notes hidden from requesters
attachments	entity_type, entity_id, storage_path, filename, mime_type, uploaded_by, size_bytes	MinIO storage path; entity_type allows attachments on cases, tasks, or any entity

Migration 005 — Integration & Notifications

Table	Key Columns	Notes
connectors	id, tenant_id, name, type (rest/webhook/kafka_producer/cron), config JSONB, trigger_config JSONB, status (active/inactive/error)	config holds type-specific settings (url, method, headers for REST; topic for Kafka; schedule for cron)
connector_logs	connector_id, triggered_by, request_payload JSONB, response_payload JSONB, status (success/error/retrying), duration_ms, error_message	Full request/response capture for debugging
notification_templates	tenant_id, slug (UNIQUE per tenant), name, channel (email/in_app/both), subject, body (Handlebars)	Reusable templates; body supports `{{variableName}}` context substitution
notifications	tenant_id, recipient_id, channel, subject, body, entity_type, entity_id, read_at, delivery_status	recipient_id → users.id FK. read_at NULL = unread. delivery_status: sent/failed/pending

Migration 006 — Audit Log

Column	Type	Description
tenant_id	UUID	Tenant partition key
entity_type	VARCHAR	e.g. 'task', 'case', 'process_instance', 'user'
entity_id	UUID	FK to the entity being acted upon
action	VARCHAR	e.g. 'created', 'completed', 'status.resolved', 'variables_updated'
actor_id	UUID (nullable)	Internal user UUID; NULL for system-generated events
before_state	JSONB	Snapshot of entity state before the action
after_state	JSONB	Snapshot of entity state after the action
ip_address	VARCHAR	Client IP (injected by gateway)

> ⚠ **Immutability Enforcement**
>
> A PostgreSQL `RULE` blocks `UPDATE` and `DELETE` on `audit_log`. Even a superuser cannot alter audit records through SQL commands — they would need to DROP the rule first, which is itself logged by Postgres activity monitoring.

4.2 API Design

All public APIs are mounted under `/api/v1/` at the gateway. Services that use `setGlobalPrefix('api')` have their internal routes prefixed; the gateway adjusts forwarded paths accordingly.

Request Pipeline

```
Client Request
  → Nginx (reverse proxy, static file serving)
  → API Gateway :3000
    [1] JwtAuthGuard   — validates Bearer token via Keycloak JWKS
    [2] TenantInterceptor — extracts tenant_id from JWT, injects X-Tenant-ID
    [3] Route Controller — matches /api/v1/{service}/{resource}
    [4] ProxyService.forward() — HTTP forward to downstream service
  → Downstream Service
    [5] Extract headers — X-Tenant-ID, X-User-ID
    [6] Business logic — DB queries, Kafka publish
    [7] Response — JSON serialised
  → API Gateway → Client
```

Complete Route Reference

Prefix	Method	Path	Action
Process Management (/api/v1/processes)			
definitions	GET/POST	/definitions	List / Create BPMN definition
	GET/PUT/DELETE	/definitions/:id	Get / Update / Delete
	POST	/definitions/:id/publish	Activate (draft → active)
	POST	/definitions/:id/unpublish	Deactivate (active → draft)
	POST	/definitions/:id/archive	Archive
instances	GET/POST	/instances	List / Start instance
	GET	/instances/:id	Get instance + BPMN XML + tasks
	PATCH	/instances/:id/suspend|resume|terminate	Lifecycle control
	PATCH	/instances/:id/variables	Merge JSONB variables
analytics	GET	/analytics/summary	KPI totals
	GET	/analytics/by-definition	Instances grouped by process
	GET	/analytics/over-time?days=30	Time-series data
	GET	/analytics/bottlenecks	Queued task nodes
	GET	/analytics/sla	SLA compliance per process
	GET	/analytics/workload	Task count per assignee
Task Management (/api/v1/tasks)			
tasks	GET	/tasks	List (filters: status, assigneeId, instanceId, caseId, overdue)
	GET	/tasks/:id	Get task detail with variables
	POST	/tasks/:id/claim	Assign to calling user
	POST	/tasks/:id/complete	Complete with outcome + variables → advances process
Cases (/api/v1/cases)			

cases	GET/POST	/cases	List (multi-filter) / Create
	GET/PUT	/cases/:id	Get / Update
	PATCH	/cases/:id/transition	State machine transition
	PATCH	/cases/:id/assign	Assign user or team
	GET/POST	/cases/:id/comments	Work notes
Approvals (/api/v1/approval)			
policies	GET/POST	/approval/policies	List / Create policy
instances	GET/POST	/approval/instances	List / Create instance
	POST	/approval/instances/:id/decide	Approve or reject step

4.3 Service Internals

BPM Orchestrator — Internal Structure

Modules

- **ProcessDefinitionModule** — CRUD for BPMN definitions, publish/unpublish/archive
- **ProcessInstanceModule** — Instance lifecycle, BPMN execution engine, variable propagation
- **TaskModule** — Task CRUD, claim/complete/escalate, SLA scheduler
- **AnalyticsModule** — 6 aggregate endpoints for dashboard metrics
- **AuditModule** — Append-only audit log writes
- **KafkaModule** — Producer service shared across all modules
- **DatabaseModule** — pg.Pool wrapper with paginate() helper

SLA Scheduler

A `@nestjs/schedule` cron job runs every 5 minutes:

```
@Cron('0 */5 * * * *')
async checkSlaBreaches() {
  SELECT tasks WHERE due_at < NOW()
    AND sla_breached = false
    AND status NOT IN ('completed','cancelled')
  → UPDATE sla_breached = true
  → Kafka: bpm.task.sla_breach
}
```

The Case Service runs the same pattern for case SLA (every 5 minutes), producing `bpm.case.sla_breach`.

Org Service — Manager Chain Resolution

The approval engine requires resolving the organisational hierarchy to find approvers at different levels (L1 = direct manager, L2 = skip-level, etc.). This is implemented via a recursive query on the materialized path:

```
-- Find manager N levels up from a user
SELECT ou.manager_id
FROM users u
JOIN user_org_assignments uoa ON uoa.user_id = u.id AND uoa.is_primary = true
JOIN org_units ou ON ou.id = uoa.org_unit_id
WHERE u.id = $userId
-- For L2+, traverse parent_id chain using the path column
```

Notification Service — Kafka Consumer

The Notification Service subscribes to multiple Kafka topics and maps each event type to a notification template:

Kafka Topic	Template Slug	Recipient
bpm.task.created	task-assigned	task.assignee_id
bpm.task.sla_breach	task-sla-breach	task.assignee_id + managers
bpm.process.started	process-started	initiator_id
bpm.approvals	approval-required	next approver in step
bpm.case.created	case-created	requester_id
bpm.case.sla_breach	case-sla-breach	assignee_id + assigned_team

4.4 BPMN Execution Engine

> **Custom Lightweight Engine**
>
> The platform implements its own BPMN execution engine (bpmn-parser.ts) without relying on Flowable, Camunda, or any external BPM runtime. This keeps the deployment self-contained and eliminates JVM dependencies.

Supported BPMN Elements

Start / End

- `startEvent`
- `endEvent`
- `intermediateThrowEvent`
- `intermediateCatchEvent`
- `boundaryEvent`

Tasks

- `userTask`
- `serviceTask`
- `scriptTask`
- `manualTask`
- `receiveTask`
- `sendTask`

Gateways

- `exclusiveGateway`
- `parallelGateway`
- `inclusiveGateway`
- `eventBasedGateway`
- `subProcess`
- `callActivity`

Execution Algorithm

```
function advance(instanceId, fromNodeId, variables) {
  el = getElementById(fromNodeId)

  if el.type == 'endEvent':
    UPDATE process_instances SET status='completed'
    Kafka → bpm.process.completed

  if el.type == 'exclusiveGateway':
    chosen = outFlows.find(f => evaluateCondition(f.condition, variables))
    advance(instanceId, chosen.target, variables)

  if el.type == 'parallelGateway' or 'inclusiveGateway':
    for each outFlow:
      if target == userTask: createTask(...)
      else: advance(instanceId, target, variables)

  if el.type == 'userTask':
    INSERT tasks(status='pending', assignee=el.assignee, ...)
    Kafka → bpm.task.created

  if el.type == 'serviceTask':
    Kafka → bpm.service.task (Integration Hub picks it up)
    advance(instanceId, nextNode, variables)    // auto-advance
```

```
  if el.type == 'startEvent' or 'intermediateEvent':
    advance(instanceId, firstOutFlow.target, variables)
}
```

Condition Evaluation

Gateway conditions use Flowable/Camunda expression language syntax, simplified to JavaScript evaluation:

```
// Input: "${changeType == 'STANDARD' && riskLevel != 'critical'}"
// Replaces ${var} with JSON.stringify(variables[var])
expr = condition
  .replace(/\$\{([^}]+)\}/g, (_, v) => JSON.stringify(variables[v]))
// Evaluates in a sandboxed Function context
return new Function(`return (${expr})`)();
```

4.5 Event-Driven Architecture (Kafka)

Kafka runs in **KRaft mode** (no Zookeeper dependency), with 3 partitions and replication factor 1 (single-broker for development). In production, a 3-broker cluster with RF=3 is recommended.

Topic Inventory

Topic	Publisher	Consumer(s)	Message Shape
`bpm.gateway.requests`	API Gateway	—	requestId, method, url, statusCode, duration, tenantId
`bpm.process.started`	BPM Orchestrator	Notification Svc	tenantId, instanceId, definitionId, initiatorId, businessKey
`bpm.process.completed`	BPM Orchestrator	—	tenantId, instanceId
`bpm.task.created`	BPM Orchestrator	Notification Svc	tenantId, instanceId, nodeId, name, assignee
`bpm.task.claimed`	BPM Orchestrator	—	tenantId, taskId, userId, instanceId
`bpm.task.completed`	BPM Orchestrator	—	tenantId, taskId, instanceId, nodeId, outcome, variables
`bpm.task.sla_breach`	BPM Orchestrator	Notification Svc	tenantId, taskId, instanceId, assigneeId, taskName, dueAt
`bpm.service.task`	BPM Orchestrator	Integration Hub	tenantId, instanceId, nodeId, serviceType, config
`bpm.approvals`	Approval Service	Notification Svc	tenantId, instanceId, status, actorId
`bpm.case.created`	Case Service	Notification Svc, Integration Hub	tenantId, caseId, caseNumber, type, priority, requesterId
`bpm.case.status_changed`	Case Service	Notification Svc	tenantId, caseId, caseNumber, from, to, actorId
`bpm.case.assigned`	Case Service	Notification Svc	tenantId, caseId, assigneeId, teamId, actorId
`bpm.case.sla_breach`	Case Service	Notification Svc	tenantId, caseId, caseNumber, type, priority, assigneeId, dueAt
`bpm.connectors.updated`	Integration Hub	—	event, connectorId, tenantId
`bpm.org.changed`	Org Service	—	tenantId, org_unit_id, action

4.6 Authentication & Multi-Tenancy

Authentication Flow

```
1. User navigates to http://localhost:8080
2. React SPA detects no session → Keycloak redirect (PKCE / S256)
3. User authenticates at Keycloak :8443
4. Keycloak issues JWT (RS256, 1h TTL) + Refresh Token (2h TTL)
5. SPA stores tokens in memory (not localStorage — XSS mitigation)
6. Every API call: Authorization: Bearer {access_token}
7. API Gateway: passport-jwt extracts + verifies token via JWKS
8. req.user = { sub, email, username, roles, tenantId }
9. TenantInterceptor: req.tenantId = payload.tenantId || 'default-tenant'
10. Downstream: X-Tenant-ID and X-User-ID headers injected
11. Services: keycloak_id lookup resolves sub → internal UUID
```

Multi-Tenancy Pattern

Aspect	Implementation
Tenant identification	JWT claim `tenant_id` or `tenantId`, fallback to default demo tenant
Data isolation	Every table has `tenant_id UUID NOT NULL`; all queries include `WHERE tenant_id=$1`
Cross-tenant prevention	No service exposes a cross-tenant query endpoint; tenant_id always passed as first parameter
Tenant provisioning	`POST /api/v1/tenants` creates tenant + default roles + admin user

4.7 Frontend Architecture

Route	Component	Key Features
/dashboard	Dashboard	KPI cards, 14-day trend chart, process snapshot, SLA alert banner
/inbox	WorkInbox	4 tabs: My Tasks, Task Pool, Approvals, Notifications. Dynamic form rendering for task completion.
/cases	CaseList	Process-driven New Case dialog: select process → parse start event form fields → create case + start instance
/cases/:id	CaseDetail	Case header, ITIL transitions, comments, timeline, assignment
/processes/:id/studio	ProcessStudio	bpmn-js Modeler + 300px PropertiesPanel (assignee, candidateGroups, slaHours, serviceType, formFields). Import from Flowable XML with auto-layout.
/processes/instances/:id	ProcessInstanceDetail	3 tabs: Diagram (token highlighting), Tasks, Variables (inline editable)
/processes/analytics	ProcessAnalytics	Recharts: bar, pie, line charts + bottleneck table, SLA compliance, workload
/org	OrgStructure	Org tree, users table, roles & positions management
/admin/connectors	ConnectorAdmin	Connector CRUD, type-specific config forms, test execution, execution logs

5. Deployment Architecture

5.1 Container Topology

Container	Image	Ports	Volumes / Persistence
bpm-postgres	postgres:15	5432	pgdata volume, migrations + seeds mounted at init
bpm-kafka	bitnami/kafka:latest	9092	kafka-data volume, KRaft mode (CLUSTER_ID auto-generated)
bpm-kafka-ui	provectuslabs/kafka-ui	8090	—
bpm-keycloak	quay.io/keycloak/keycloak:24	8443	realm-export.json mounted at import path
bpm-minio	minio/minio	9000, 9001	minio-data volume
bpm-prometheus	prom/prometheus	9090	prometheus.yml config
bpm-grafana	grafana/grafana	3300	grafana-data volume
bpm-api-gateway	infra-api-gateway	3000	—
bpm-org-service	infra-org-service	3001	—
bpm-approval-service	infra-approval-service	3002	—
bpm-orchestrator	infra-bpm-orchestrator	3003	—
bpm-case-service	infra-case-service	3004	—
bpm-integration-hub	infra-integration-hub	3005	—
bpm-notification-service	infra-notification-service	3006	—
bpm-frontend	infra-frontend (nginx:1.25)	8080	—

5.2 Network & Dependencies

All containers share a single Docker bridge network (`bpm-network`). Service discovery is by container name (DNS). The startup dependency chain ensures infrastructure services are healthy before application services start:

```
postgres + kafka ──► api-gateway, org-service, approval-service,
                     bpm-orchestrator, case-service, integration-hub,
                     notification-service
keycloak         ──► api-gateway (JWKS endpoint readiness)
frontend         ──► api-gateway (healthcheck dependency)
```

5.3 Environment Configuration

Variable	Value (Docker Compose)	Purpose
DATABASE_URL	postgres://bpm:bpm_pass_2024@postgres:5432/bpm_db	pg.Pool connection string
KAFKA_BROKERS	kafka:9092	KafkaJS broker list
JWT_PUBLIC_KEY_URL	http://keycloak:8080/realms/bpm/…/certs	JWKS endpoint for token validation
MINIO_ENDPOINT	minio	Object storage host
NODE_ENV	production	NestJS production mode

5.4 Startup Sequence

```
cd /opt/bpm/infra
docker compose up -d --build

# Health check sequence (~2 min):
1. postgres     → accepts connections
2. kafka        → controller elected (KRaft)
3. keycloak     → realm "bpm" imported, /realms/bpm available
4. minio        → bucket bpm-attachments created
5. all services → DB connected, Kafka consumer subscribed
6. api-gateway  → JWKS fetched from Keycloak
7. frontend     → nginx serving on :8080
```

5.5 Production Considerations

⚠ **Development vs Production Differences**

- Replace single-broker Kafka with 3-broker cluster (RF=3)
- Separate PostgreSQL per service (or use schemas + RLS)
- TLS termination at load balancer (nginx upstream)
- Keycloak clustered (Active/Active or Active/Standby)
- MinIO distributed mode for HA
- Secrets management via Vault or k8s Secrets (not env vars)

☁ **Cloud Migration Path**

- Each NestJS service → Kubernetes Deployment + HPA
- PostgreSQL → RDS (managed) or Cloud SQL
- Kafka → MSK (AWS) or Confluent Cloud
- Keycloak → managed OIDC (Auth0, Okta) or EKS Keycloak
- MinIO → S3 (AWS) or GCS
- nginx → ALB / Cloud Load Balancer

6. Operational Guide

6.1 Access Points

System	URL	Default Credentials
Frontend Portal	http://localhost:8080	admin / Admin123! (or any demo user)
API Gateway (Swagger)	http://localhost:3000/api/docs	JWT required
Keycloak Admin	http://localhost:8443/admin	admin / Admin123!
Kafka UI	http://localhost:8090	—
MinIO Console	http://localhost:9001	minioadmin / minioadmin2024
Prometheus	http://localhost:9090	—
Grafana	http://localhost:3300	admin / admin

6.2 Demo Users

Username	Password	Role	Org Unit	Typical Use
admin	Admin123!	admin	Demo Corp (CEO)	All administration, configuration
requester1	Admin123!	requester	Infrastructure Team	Submit requests, raise incidents
manager1	Admin123!	manager	IT Department (Manager)	Approve requests, manage team
finance1	Admin123!	finance_controller	Finance Dept	Financial approval steps
cab1	Admin123!	cab_member	Finance Dept	Change Advisory Board reviews
engineer1	Admin123!	it_engineer	Infrastructure Team	Task execution, case resolution

6.3 Common Operations

Create a Process

1. Navigate to **Processes → New Definition**
2. Open the BPMN Studio — drag tasks, gateways from palette
3. Click each user task → set assignee / candidateGroups / SLA hours / form fields in Properties Panel
4. Set sequence flow conditions on gateway outflows (e.g. `${outcome == 'approved'}`)
5. Save → **Publish** to make it startable

Raise a Case with Process

1. Navigate to **Cases → New Case**
2. Select an active process from the dropdown — form fields from the start event render dynamically
3. Fill in title, priority, assigned team, and process-specific fields
4. Submit — case is created (INC/CHG/REQ number assigned) and process instance started
5. Case assignees receive in-app notifications

Work Inbox

1. Navigate to **Inbox**
2. **Task Pool** — unassigned tasks; click *Claim* to take ownership

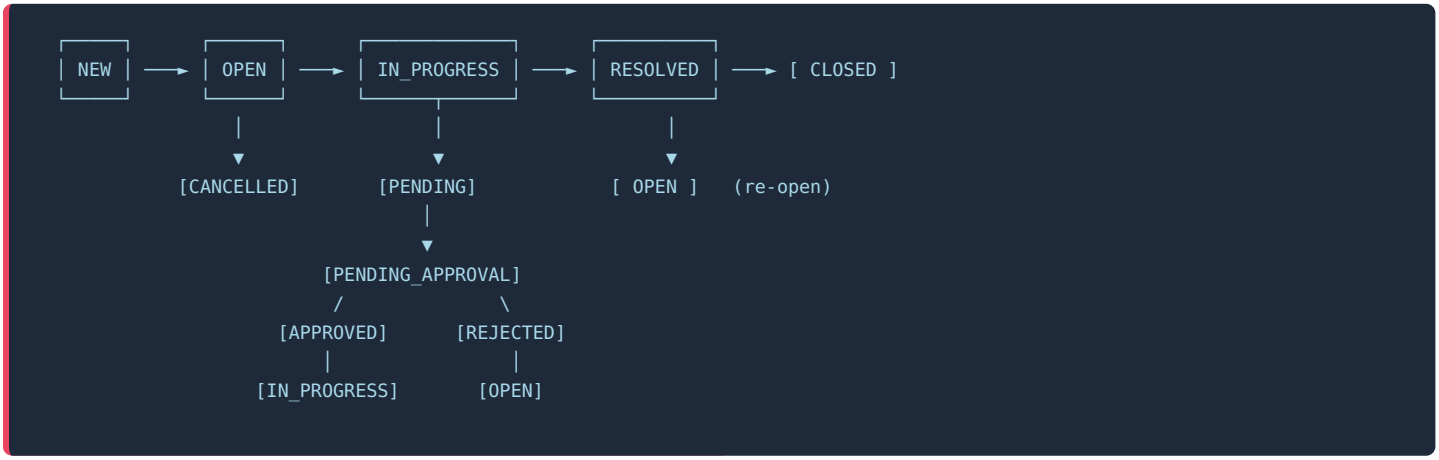
3. **My Tasks** — claimed tasks; click *Complete* to fill form + set outcome
4. Outcome drives exclusive gateway branching (e.g. `approved` vs `rejected`)

6.4 Key Rebuild Commands

```
cd /opt/bpm/infra

# Full rebuild
docker compose up -d --build

# Rebuild a single service
docker compose build <service-name>
docker compose up -d <service-name>

# View logs
docker logs bpm-<service> -f --tail=50

# DB console
docker exec -it bpm-postgres psql -U bpm -d bpm_db

# Reset all data (keeps images)
docker compose down -v
docker compose up -d --build
```

7. Appendix

Appendix A — ITIL Case State Machine



Appendix B — Form Field Types

BPMN Attribute / Element	Rendered Widget	Storage Format
type="string"	Text input	String
type="textarea"	Multi-line textarea	String
type="long" / "integer" / "double"	Number input	Number
type="date"	Date picker	ISO 8601 string
type="boolean"	Checkbox	Boolean
type="enum"	Select dropdown	String (selected value ID)

Form fields are stored as URL-encoded JSON on the `camunda:formFields` attribute (bpmn-js native) or parsed from `<flowable:formProperty>` children (Flowable XML import). The `_formSchema` key is injected into task variables at creation time for the Work Inbox to render dynamically.

Appendix C — SLA Matrix

Type \ Priority	Critical	High	Medium	Low
Incident	1 hour	4 hours	8 hours	24 hours
Problem	4 hours	8 hours	24 hours	72 hours
Change	8 hours	24 hours	72 hours	168 hours
Request	4 hours	8 hours	24 hours	72 hours
Alarm	1 hour	2 hours	4 hours	8 hours

Appendix D — Glossary

Term	Definition
BPMN 2.0	Business Process Model and Notation — an industry-standard graphical notation for business processes

KRaft	Kafka Raft — Kafka's built-in consensus protocol replacing Zookeeper
OIDC	OpenID Connect — identity layer on top of OAuth2 for authentication
JWKS	JSON Web Key Set — endpoint exposing public keys for JWT signature verification
ITIL	IT Infrastructure Library — framework of best practices for IT service management
Materialized Path	A tree storage pattern storing the full ancestor path as a string for efficient hierarchy queries
Exclusive Gateway	XOR gateway — only one outgoing path is taken based on condition evaluation
Parallel Gateway	AND gateway — all outgoing paths execute simultaneously
SLA	Service Level Agreement — commitment to resolve/complete within a defined time period
Business Key	A human-readable identifier linking a process instance to a business record (e.g. case number)